

0.1 Thiết kế kiến trúc

Chương 3 đã trình bày các nền tảng lý thuyết và công nghệ được sử dụng trong quá trình xây dựng hệ thống, bao gồm công nghệ phát triển máy chủ, ứng dụng di động, cơ sở dữ liệu và thiết bị IoT. Chương này trình bày quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống chống trộm xe IoT.

Thiết kế kiến trúc là bước quan trọng nhằm xác định cách tổ chức tổng thể của hệ thống và mối quan hệ giữa các thành phần chính. Đối với hệ thống trong đề án, kiến trúc cần đảm bảo sự phối hợp giữa thiết bị IoT gắn trên phương tiện, máy chủ xử lý trung tâm, cơ sở dữ liệu và ứng dụng di động của người dùng. Việc thiết kế kiến trúc rõ ràng giúp hệ thống dễ triển khai, dễ bảo trì và thuận tiện cho việc mở rộng trong tương lai.

Trong mục này, đề án trước hết trình bày lựa chọn kiến trúc phần mềm được sử dụng cho hệ thống. Tiếp theo, kiến trúc tổng quan của hệ thống được mô tả nhằm làm rõ vai trò của từng thành phần và luồng trao đổi dữ liệu giữa các thành phần. Cuối cùng, mục này trình bày thiết kế chi tiết các gói chức năng chính, làm cơ sở cho việc triển khai server, ứng dụng di động và thiết bị IoT trong các phần tiếp theo.

0.1.1 Lựa chọn kiến trúc phần mềm

Đối với hệ thống chống trộm xe IoT, đề án lựa chọn kiến trúc client-server kết hợp mô hình IoT nhiều tầng. Kiến trúc này phù hợp với hệ thống do các thành phần tham gia có vai trò khác nhau, bao gồm thiết bị IoT gắn trên phương tiện, máy chủ xử lý trung tâm, cơ sở dữ liệu và ứng dụng di động của người dùng. Việc tổ chức hệ thống theo các tầng giúp tách biệt nhiệm vụ giữa thu thập dữ liệu, xử lý nghiệp vụ, lưu trữ dữ liệu và hiển thị thông tin cho người dùng.

Trong kiến trúc được lựa chọn, tầng thiết bị chịu trách nhiệm thu thập dữ liệu từ phần cứng, bao gồm dữ liệu cảm biến, trạng thái thiết bị và thông tin vị trí. Thiết bị IoT gửi dữ liệu về máy chủ thông qua giao thức MQTT. Giao thức này phù hợp với hệ thống IoT do hỗ trợ truyền dữ liệu nhẹ, phù hợp với các thiết bị nhúng và các tình huống cần gửi dữ liệu thường xuyên từ thiết bị lên hệ thống trung tâm.

Tầng máy chủ được xây dựng bằng NestJS và đóng vai trò trung gian giữa thiết bị IoT, cơ sở dữ liệu và ứng dụng di động. Máy chủ tiếp nhận dữ liệu từ thiết bị thông qua module MQTT, xử lý dữ liệu cảm biến, phát hiện nguy cơ trộm, lưu trữ dữ liệu và gửi cảnh báo đến người dùng khi cần thiết. Bên cạnh đó, máy chủ cung cấp các REST API để ứng dụng di động thực hiện các chức năng như đăng nhập, quản lý tài khoản, kích hoạt thiết bị, xem trạng thái thiết bị và xem lịch sử dữ liệu.

Ngoài REST API, hệ thống còn sử dụng WebSocket để hỗ trợ cập nhật dữ liệu trạng thái theo thời gian thực cho ứng dụng di động.

Dữ liệu của hệ thống được lưu trữ trong PostgreSQL và được truy cập thông qua Prisma ORM. Việc sử dụng Prisma giúp thao tác với cơ sở dữ liệu rõ ràng hơn, đồng thời tạo lớp trung gian giữa mã nguồn server và hệ quản trị cơ sở dữ liệu. Các dữ liệu chính được lưu trữ bao gồm thông tin người dùng, thông tin thiết bị, dữ liệu cảm biến, dữ liệu vị trí và lịch sử cảnh báo.

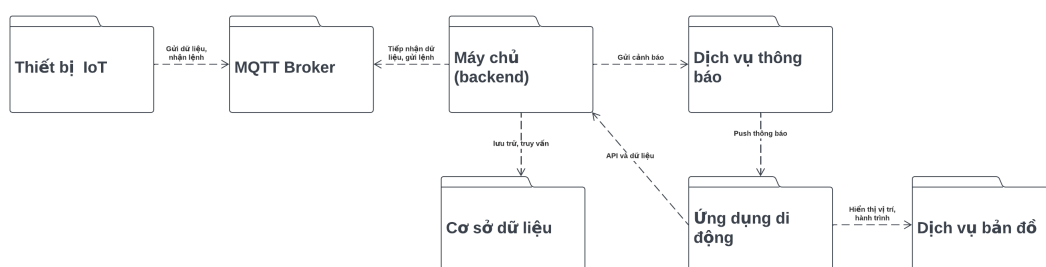
Tầng ứng dụng di động được xây dựng bằng Flutter và Dart, đóng vai trò là giao diện tương tác trực tiếp với người dùng. Ứng dụng gọi các REST API do server cung cấp để thực hiện các chức năng nghiệp vụ và sử dụng WebSocket để nhận các cập nhật trạng thái cần hiển thị kịp thời. Trong quá trình tổ chức mã nguồn, ứng dụng được xây dựng theo mô hình MVC/MVVM đơn giản nhằm tách biệt tương đối giữa giao diện, dữ liệu và xử lý logic. Đối với chức năng bản đồ, ứng dụng sử dụng OpenStreetMap để hiển thị vị trí phương tiện cho người dùng.

Ở phía server, mã nguồn được tổ chức thành các module chức năng như auth, user, device, mqtt, sensor-data, firebase và middleware. Module auth phụ trách xác thực và đăng nhập người dùng. Module user xử lý thông tin tài khoản. Module device quản lý thông tin thiết bị và quá trình kích hoạt thiết bị. Module mqtt tiếp nhận dữ liệu từ thiết bị IoT. Module sensor-data xử lý và lưu trữ dữ liệu cảm biến. Module firebase hỗ trợ gửi thông báo cảnh báo đến ứng dụng di động. Module middleware hỗ trợ xử lý các bước trung gian như kiểm tra xác thực và kiểm soát truy cập.

0.1.2 Thiết kế tổng quan

0.1.3 Thiết kế tổng quan hệ thống

Sau khi lựa chọn kiến trúc client-server kết hợp mô hình IoT nhiều tầng, hệ thống được thiết kế tổng quan thành các gói thành phần chính nhằm tách biệt vai trò và trách nhiệm của từng khối chức năng. Biểu đồ gói tổng quan của hệ thống được trình bày trong Hình 0.1.



Hình 0.1: Biểu đồ gói tổng quan của hệ thống

Như thể hiện trong Hình 0.1, hệ thống được chia thành bảy gói chính gồm: Thiết bị IoT, Ứng dụng di động, Hệ thống máy chủ, Cơ sở dữ liệu, Dịch vụ MQTT, Dịch vụ thông báo và Dịch vụ bản đồ. Mỗi gói đảm nhiệm một vai trò riêng trong quá trình thu thập dữ liệu, xử lý nghiệp vụ, lưu trữ dữ liệu và hiển thị thông tin cho người dùng.

Gói Thiết bị IoT đại diện cho thiết bị phần cứng được gắn trên phương tiện. Thành phần này có nhiệm vụ thu thập dữ liệu cảm biến, dữ liệu vị trí và trạng thái hoạt động của xe. Thiết bị IoT không giao tiếp trực tiếp với ứng dụng di động mà gửi dữ liệu về hệ thống thông qua Dịch vụ MQTT. Ngoài việc gửi dữ liệu, thiết bị cũng có thể nhận các lệnh điều khiển hoặc cấu hình từ hệ thống máy chủ thông qua kênh MQTT.

Gói Dịch vụ MQTT đóng vai trò trung gian truyền thông giữa Thiết bị IoT và Hệ thống máy chủ. Việc tách riêng dịch vụ MQTT giúp quá trình trao đổi dữ liệu giữa thiết bị và server trở nên linh hoạt hơn, phù hợp với đặc thù của hệ thống IoT, trong đó thiết bị cần gửi dữ liệu định kỳ hoặc gửi dữ liệu khi phát hiện trạng thái bất thường.

Gói Hệ thống máy chủ là thành phần xử lý trung tâm của hệ thống. Máy chủ tiếp nhận dữ liệu từ Dịch vụ MQTT, xử lý dữ liệu cảm biến, cập nhật trạng thái thiết bị, phát hiện nguy cơ trộm và cung cấp các API cho ứng dụng di động. Ngoài ra, máy chủ còn thực hiện truy vấn và lưu trữ dữ liệu thông qua gói Cơ sở dữ liệu, đồng thời kết nối với Dịch vụ thông báo để gửi cảnh báo đến người dùng khi cần thiết.

Gói Cơ sở dữ liệu có nhiệm vụ lưu trữ các dữ liệu của hệ thống, bao gồm thông tin người dùng, thông tin thiết bị, dữ liệu cảm biến, dữ liệu vị trí và lịch sử cảnh báo. Hệ thống máy chủ là thành phần trực tiếp thực hiện các thao tác lưu trữ và truy vấn dữ liệu, trong khi các thành phần khác không truy cập trực tiếp vào cơ sở dữ liệu. Cách tổ chức này giúp kiểm soát dữ liệu tập trung và đảm bảo tính nhất quán trong quá trình xử lý.

Gói Ứng dụng di động là thành phần giao tiếp trực tiếp với người dùng. Ứng dụng kết nối với Hệ thống máy chủ thông qua API và dữ liệu thời gian thực để thực hiện các chức năng như đăng nhập, quản lý thiết bị, xem trạng thái xe, xem lịch sử dữ liệu và nhận cập nhật trạng thái. Khi người dùng cần theo dõi vị trí hoặc hành trình của phương tiện, ứng dụng sử dụng Dịch vụ bản đồ để hiển thị thông tin vị trí một cách trực quan.

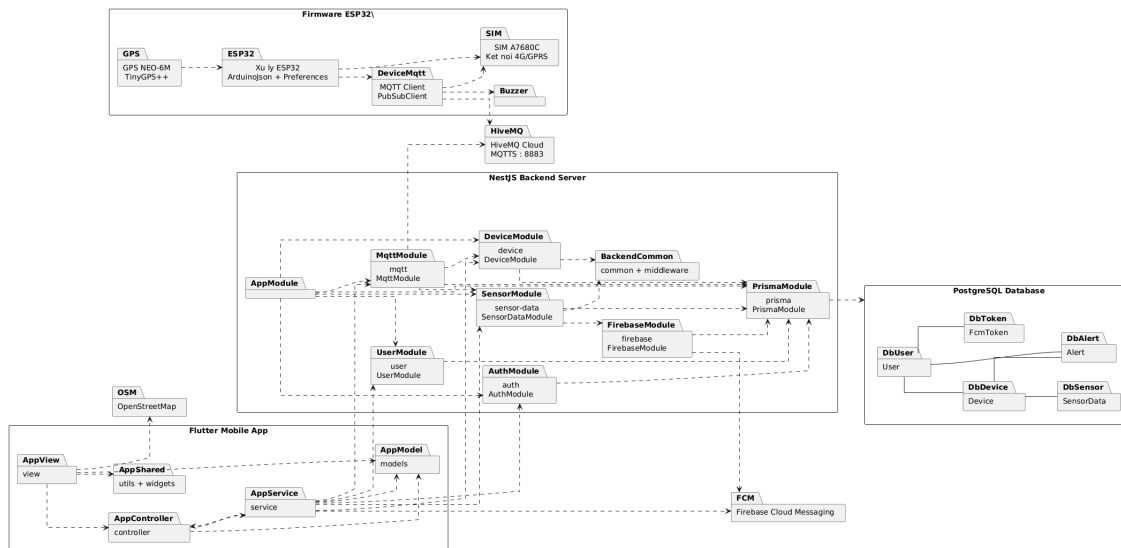
Gói Dịch vụ thông báo được sử dụng để gửi cảnh báo đến ứng dụng di động khi hệ thống phát hiện dấu hiệu bất thường. Khi Hệ thống máy chủ xác định có nguy cơ trộm, máy chủ sẽ gửi yêu cầu đến Dịch vụ thông báo. Sau đó, dịch vụ này thực

hiện gửi push notification đến thiết bị di động của người dùng. Cơ chế này giúp người dùng nhận cảnh báo kịp thời ngay cả khi không trực tiếp mở ứng dụng.

Gói Dịch vụ bản đồ hỗ trợ ứng dụng di động trong việc hiển thị vị trí và hành trình của phương tiện. Trong đồ án, dịch vụ bản đồ được sử dụng để trực quan hóa dữ liệu vị trí do thiết bị IoT gửi về, giúp người dùng dễ dàng theo dõi vị trí hiện tại hoặc vị trí gần nhất của xe.

0.1.4 Thiết kế chi tiết gói

Sau khi xác định kiến trúc tổng quan của hệ thống, đồ án tiếp tục thiết kế chi tiết các gói thành phần nhằm làm rõ cách tổ chức bên trong từng khối. Biểu đồ gói chi tiết của hệ thống được trình bày trong Hình 0.2. Biểu đồ này thể hiện các gói chính trong firmware ESP32, ứng dụng di động Flutter, backend NestJS, cơ sở dữ liệu PostgreSQL và các dịch vụ bên ngoài được sử dụng trong hệ thống.



Hình 0.2: Biểu đồ gói chi tiết của hệ thống

Như thể hiện trong Hình 0.2, hệ thống được chia thành bốn nhóm thành phần chính gồm firmware ESP32, ứng dụng di động Flutter, backend server NestJS và cơ sở dữ liệu PostgreSQL. Bên cạnh đó, hệ thống còn sử dụng một số dịch vụ bên ngoài như HiveMQ Cloud, Firebase Cloud Messaging và OpenStreetMap để hỗ trợ truyền dữ liệu, gửi thông báo và hiển thị bản đồ.

Gói *Firmware ESP32* đại diện cho chương trình nhúng được triển khai trên thiết bị IoT gắn trên phương tiện. Gói xử lý ESP32 sử dụng ArduinoJson và Preferences để xử lý dữ liệu, đóng gói thông tin và lưu một số cấu hình cần thiết trên thiết bị. Gói MQTT Client sử dụng thư viện PubSubClient để kết nối tới HiveMQ Cloud, gửi dữ liệu cảm biến, dữ liệu vị trí và nhận lệnh điều khiển từ hệ thống.

Gói *Flutter Mobile App* là thành phần giao tiếp trực tiếp với người dùng. Ứng dụng được tổ chức theo các gói nhỏ gồm view, controller, service, models, utils và widgets. Gói view chứa các màn hình giao diện như màn hình đăng nhập, quản lý thiết bị, xem trạng thái, xem lịch sử và bản đồ. Gói controller chịu trách nhiệm xử lý logic trung gian giữa giao diện và dữ liệu. Gói service thực hiện các thao tác giao tiếp với backend thông qua API, nhận dữ liệu thời gian thực và xử lý các dịch vụ liên quan. Gói models định nghĩa các lớp dữ liệu được sử dụng trong ứng dụng, chẳng hạn như người dùng, thiết bị, dữ liệu cảm biến và cảnh báo. Các thành phần dùng chung như hàm tiện ích và widget tái sử dụng được đặt trong gói utils và widgets. Cách tổ chức này giúp ứng dụng tách biệt tương đối giữa giao diện, xử lý logic và dữ liệu.

Gói *NestJS Backend Server* là thành phần xử lý trung tâm của hệ thống. App-Module đóng vai trò module gốc, liên kết các module chức năng chính của backend. AuthModule chịu trách nhiệm xử lý đăng nhập, xác thực người dùng và kiểm soát quyền truy cập. UserModule quản lý thông tin tài khoản người dùng. DeviceModule xử lý các chức năng liên quan đến thiết bị như kích hoạt thiết bị, cập nhật thông tin thiết bị và truy xuất danh sách thiết bị của người dùng. MqttModule đảm nhiệm việc kết nối tới HiveMQ Cloud để tiếp nhận dữ liệu do thiết bị IoT gửi lên và gửi lệnh điều khiển đến thiết bị khi cần thiết. SensorDataModule xử lý dữ liệu cảm biến, dữ liệu vị trí, cập nhật trạng thái thiết bị và phát hiện các dấu hiệu bất thường. FirebaseModule hỗ trợ gửi thông báo cảnh báo đến ứng dụng di động thông qua Firebase Cloud Messaging. PrismaModule đóng vai trò lớp truy cập dữ liệu, cung cấp các thao tác làm việc với cơ sở dữ liệu PostgreSQL. Ngoài ra, gói common và middleware chứa các thành phần dùng chung như hàm tiện ích, xử lý xác thực, kiểm tra dữ liệu và các xử lý trung gian khác.

Gói *PostgreSQL Database* lưu trữ các thực thể dữ liệu chính của hệ thống. Bảng User lưu thông tin tài khoản người dùng. Bảng Device lưu thông tin thiết bị và phương tiện được liên kết với người dùng. Bảng SensorData lưu các dữ liệu cảm biến, vị trí và trạng thái được gửi từ thiết bị IoT. Bảng Alert lưu lịch sử các cảnh báo được hệ thống tạo ra khi phát hiện dấu hiệu bất thường. Bảng FcmToken lưu token thông báo của thiết bị di động để phục vụ việc gửi push notification. Các quan hệ dữ liệu chính gồm một người dùng có thể quản lý nhiều thiết bị, một thiết bị có nhiều bản ghi dữ liệu cảm biến, một thiết bị có thể phát sinh nhiều cảnh báo và một người dùng có thể có nhiều token thông báo.

Các dịch vụ bên ngoài được tách thành các gói riêng để thể hiện rõ vai trò hỗ trợ của chúng trong hệ thống. HiveMQ Cloud được sử dụng làm MQTT broker, đóng

vai trò trung gian truyền dữ liệu giữa thiết bị IoT và backend. Firmware ESP32 gửi dữ liệu lên HiveMQ Cloud thông qua MQTT, trong khi MqttModule của backend đăng ký nhận dữ liệu từ broker để xử lý. Firebase Cloud Messaging được sử dụng để gửi thông báo cảnh báo đến ứng dụng di động khi hệ thống phát hiện nguy cơ trộm. OpenStreetMap được ứng dụng di động sử dụng để hiển thị vị trí và hành trình của phương tiện trên bản đồ.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

a, Mục tiêu thiết kế giao diện

Ứng dụng di động được thiết kế hướng tới người dùng sử dụng điện thoại Android với kích thước màn hình từ nhỏ đến trung bình lớn. Giao diện chủ yếu được thiết kế theo hướng màn hình dọc, phù hợp với thói quen sử dụng điện thoại và các thao tác như xem trạng thái thiết bị, nhận cảnh báo, theo dõi vị trí phương tiện.

Về kích thước màn hình, ứng dụng hướng tới các thiết bị Android phổ biến có kích thước khoảng từ 5 inch đến 6.7 inch. Về độ phân giải, giao diện được thiết kế để hiển thị tốt trên các thiết bị có độ phân giải từ HD+ đến Full HD+. Các thành phần giao diện như văn bản, nút bấm, biểu tượng, thẻ thông tin và bản đồ cần được bố trí linh hoạt để hạn chế tình trạng tràn nội dung hoặc khó thao tác trên màn hình nhỏ.

Ứng dụng hỗ trợ hiển thị trên màn hình màu của điện thoại thông minh. Giao diện sử dụng màu sắc đơn giản, có màu chủ đạo cho các thành phần chính và màu nhấn để thể hiện trạng thái thiết bị hoặc cảnh báo. Các trạng thái quan trọng như bình thường, mất kết nối hoặc cảnh báo cần được thể hiện rõ ràng để người dùng dễ nhận biết.

b, Quy chuẩn thiết kế giao diện

Để đảm bảo tính thống nhất và dễ sử dụng, ứng dụng áp dụng một số quy chuẩn chung trong quá trình thiết kế giao diện. Các màn hình được thiết kế với lề ngang 16px, khoảng cách giữa các thành phần sử dụng theo các mức 8px, 12px, 16px và 24px. Các card và nút chức năng chủ yếu sử dụng bán kính bo góc 8px nhằm tạo cảm giác mềm mại và dễ nhìn. Các vùng tương tác như nút bấm và biểu tượng được thiết kế với kích thước chạm tối thiểu 48×48px để thuận tiện cho thao tác trên màn hình cảm ứng.

Tiêu đề của mỗi màn hình được đặt ở phía bên trái của AppBar, trong khi các thao tác phụ như làm mới, chỉnh sửa hoặc mở rộng bản đồ được đặt ở phía bên phải. Điều hướng chính của ứng dụng sử dụng thanh điều hướng dưới, bao gồm các mục

Trang chủ, Thiết bị, Cảnh báo và Tài khoản. Đối với các nội dung dài như danh sách dữ liệu cảm biến, lịch sử cảnh báo hoặc lịch sử hành trình, ứng dụng sử dụng cơ chế cuộn hoặc phân trang, tránh tải toàn bộ dữ liệu lên giao diện cùng một lúc.

Các nút và điều khiển trong ứng dụng được sử dụng theo từng mục đích cụ thể. *FilledButton* được dùng cho các thao tác chính như lưu, xác nhận hoặc kích hoạt thiết bị. *OutlinedButton* được dùng cho các thao tác phụ như bật bảo vệ hoặc điều khiển còi. *TextButton* được dùng cho các thao tác ít quan trọng như hủy hoặc quay lại. *IconButton* được dùng cho các thao tác nhanh như làm mới, chỉnh sửa, mở rộng bản đồ hoặc xóa bộ lọc. Đối với các thao tác nguy hiểm như xóa thiết bị, giao diện sử dụng màu đỏ và hiển thị hộp thoại xác nhận trước khi thực hiện. Trong quá trình gửi yêu cầu lên hệ thống, nút thao tác được vô hiệu hóa và hiển thị biểu tượng tải để tránh người dùng gửi lặp lại nhiều lần.

Về màu sắc, ứng dụng định nghĩa màu được sử dụng như sau. Màu xanh dương (#2563EB) được sử dụng cho các thành phần chính, liên kết và vị trí. Màu xanh lá (#16A34A) biểu thị trạng thái an toàn hoặc đang bảo vệ. Màu vàng cam (#F59E0B) biểu thị trạng thái cảnh báo hoặc chưa được bảo vệ. Màu đỏ (#DC2626) được dùng cho trạng thái nguy hiểm, mất cấp hoặc lỗi. Bên cạnh đó, ứng dụng sử dụng màu (#111827) cho văn bản chính, (#6B7280) cho văn bản phụ, (#F6F8FB) cho nền sáng, (#111827) cho nền tối và (#1F2937) cho bề mặt tối. Khi biểu diễn trạng thái, màu sắc luôn đi kèm biểu tượng hoặc nội dung chữ, không sử dụng màu làm dấu hiệu duy nhất.

Về phân cấp chữ, tiêu đề màn hình sử dụng cỡ chữ khoảng 20px và kiểu chữ đậm. Tiêu đề card hoặc khu vực sử dụng cỡ chữ 17–18px. Nội dung chính sử dụng cỡ chữ 13–16px. Các thông tin phụ như thời gian, tọa độ hoặc mô tả ngắn sử dụng cỡ chữ 11–12px và màu chữ phụ. Với các nội dung dài, ứng dụng giới hạn số dòng hiển thị và sử dụng dấu ba chấm để tránh làm rối giao diện.

Bố cục chung của các màn hình được tổ chức theo thứ tự gồm AppBar, vùng thông tin tổng quan, khu vực tìm kiếm hoặc bộ lọc, nội dung chính, trạng thái phân trang hoặc trạng thái trống và thanh điều hướng dưới. Ứng dụng hỗ trợ cả giao diện sáng và tối, trong đó ý nghĩa màu sắc, độ tương phản và vị trí các điều khiển chính được giữ nhất quán giữa hai chế độ.

c, Một số hình ảnh thiết kế

Thanh điều hướng dưới : Thanh điều hướng dưới được thiết kế để giúp người dùng di chuyển nhanh giữa các chức năng chính của ứng dụng. Như minh họa trong Hình 0.3, thanh điều hướng gồm bốn mục: Trang chủ, Thiết bị, Cảnh báo và User. Mỗi mục bao gồm biểu tượng và nhãn văn bản để người dùng dễ nhận biết chức

năng tương ứng.

Mục đang được chọn được làm nổi bật bằng màu sắc hoặc nền khác biệt, giúp người dùng biết được vị trí hiện tại trong ứng dụng. Cách thiết kế này giúp việc điều hướng giữa các màn hình chính trở nên đơn giản, trực quan và nhất quán.



Hình 0.3: Thiết kế thanh điều hướng dưới của ứng dụng

Màn hình trang chủ : Màn hình trang chủ được thiết kế nhằm cung cấp cái nhìn tổng quan về trạng thái các phương tiện mà người dùng đang quản lý. Như minh họa trong Hình 0.4, phần đầu màn hình hiển thị khu vực thông báo chung cho tất cả các xe, bao gồm các chỉ số tổng quan như tổng số xe, số xe đang an toàn và số xe cần kiểm tra. Cách bố trí này giúp người dùng nhanh chóng nắm được tình trạng chung của hệ thống ngay khi mở ứng dụng.

Bên dưới khu vực tổng quan là phần “Xe cần kiểm tra”, dùng để hiển thị các phương tiện đang có trạng thái bất thường hoặc cần người dùng chú ý. Tiếp theo là phần “Cảnh báo gần đây”, hiển thị các cảnh báo mới phát sinh trong hệ thống. Các thông tin này được trình bày theo dạng thẻ nhằm giúp nội dung dễ quan sát và thuận tiện khi mở rộng thêm chi tiết.

Phía dưới màn hình là thanh điều hướng chính, cho phép người dùng chuyển nhanh sang các nhóm chức năng khác như thiết bị, cảnh báo và tài khoản. Cách bố trí này giúp màn hình trang chủ vừa đóng vai trò tổng quan, vừa là điểm truy cập nhanh đến các chức năng quan trọng của ứng dụng.

Thông báo chung cho tất cả các xe

Tổng số

An toàn

Cần kiểm tra

Xe cần kiểm tra

Cảnh báo gần đây

AppBar

Hình 0.4: Thiết kế giao diện màn hình trang chủ

Màn hình chi tiết thiết bị : Màn hình chi tiết thiết bị được thiết kế để hiển thị thông tin và trạng thái hoạt động của một thiết bị cụ thể. Như minh họa trong Hình 0.5, phần đầu màn hình hiển thị biển số xe, mã thiết bị và ngày kích hoạt, giúp người dùng nhận biết nhanh phương tiện đang được theo dõi.

Bên dưới là các nút thao tác chính gồm bật/tắt chế độ bảo vệ, bật/tắt còi và xóa thiết bị. Khu vực hành trình hiển thị bản đồ vị trí xe, kèm các lựa chọn lọc theo khoảng thời gian như 24 giờ, 7 ngày hoặc tất cả. Phần dữ liệu cảm biến cho phép người dùng lọc theo ngày và xem các thông tin như tốc độ, vị trí và thời gian cập nhật.

Chi tiết thiết bị

Settings

Biển số xe

Mã thiết bị

Ngày kích hoạt

Bật/ Tắt bảo vệ

Bật/Tắt còi

Hành trình

24 giờ

7 ngày

Tất cả

Map

Dữ liệu cảm biến

Lọc theo ngày

Tốc độ

Vị trí

Thời gian

Thanh điều hướng dưới

Hình 0.5: Thiết kế giao diện màn hình chi tiết thiết bị

Màn hình cảnh báo : Màn hình cảnh báo được thiết kế để hiển thị các thông báo bất thường phát sinh từ hệ thống. Như minh họa trong Hình 0.6, phần đầu màn hình hiển thị tổng số cảnh báo và cảnh báo mới nhất, giúp người dùng nhanh chóng nắm bắt tình hình chung.

Bên dưới là các công cụ lọc và tìm kiếm, bao gồm lọc theo ngày, lọc theo thiết bị và ô tìm kiếm. Danh sách cảnh báo được hiển thị dưới dạng thẻ, trong đó mỗi thẻ gồm biển số xe, thời gian, nội dung cảnh báo, mã thiết bị và dữ liệu liên quan. Cách bố trí này giúp người dùng dễ dàng tra cứu, phân loại và xem lại các cảnh báo đã phát sinh.

Cảnh báo

Settings

Tổng số cảnh báo

Mới nhất

Lọc theo ngày

Lọc theo thiết bị

Tìm kiếm

Biến số

Thời gian

Message

Mã thiết bị

Dữ liệu

Thanh điều hướng dưới

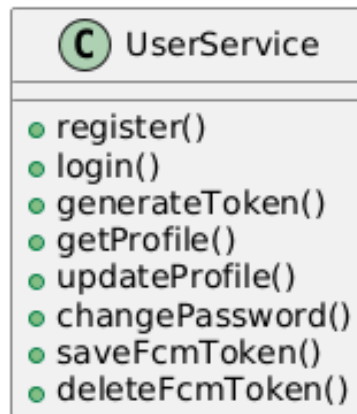
Hình 0.6: Thiết kế giao diện màn hình cảnh báo

0.2.2 Thiết kế lớp

a, Lớp UserService

Lớp *UserService* chịu trách nhiệm xử lý các nghiệp vụ liên quan đến người dùng trong hệ thống. Đây là lớp trung gian giữa controller và cơ sở dữ liệu, thực hiện các thao tác như đăng ký tài khoản, đăng nhập, tạo token xác thực, truy xuất và cập nhật thông tin cá nhân của người dùng.

Class UserService



Hình 0.7: Thiết kế lớp UserService

Phương thức *register()* được sử dụng để tạo tài khoản người dùng mới. Phương thức *login()* xử lý quá trình đăng nhập và kiểm tra thông tin xác thực của người dùng. Sau khi đăng nhập thành công, phương thức *generateToken()* được sử dụng để sinh token phục vụ cho việc xác thực các yêu cầu tiếp theo từ ứng dụng di động.

Bên cạnh các chức năng xác thực, *UserService* còn cung cấp các phương thức quản lý thông tin cá nhân. Phương thức *getProfile()* dùng để lấy thông tin tài khoản hiện tại của người dùng, trong khi *updateProfile()* cho phép cập nhật các thông tin cá nhân. Phương thức *changePassword()* được sử dụng khi người dùng muốn thay đổi mật khẩu tài khoản.

Ngoài ra, lớp này còn hỗ trợ quản lý token thông báo của thiết bị di động thông qua hai phương thức *saveFcmToken()* và *deleteFcmToken()*. Các phương thức này giúp hệ thống lưu hoặc xóa token Firebase Cloud Messaging, phục vụ cho chức năng gửi cảnh báo đến đúng thiết bị của người dùng.

b, Lớp DeviceService

Lớp *DeviceService* chịu trách nhiệm xử lý các nghiệp vụ liên quan đến thiết bị chống trộm trong hệ thống. Đây là lớp trung gian giữa các yêu cầu từ ứng dụng di động, dữ liệu thiết bị trong cơ sở dữ liệu và các lệnh điều khiển gửi đến thiết bị IoT.

Class DeviceService



Hình 0.8: Thiết kế lớp `DeviceService`

Phương thức `createDevice()` được sử dụng để tạo mới thông tin thiết bị trong hệ thống. Phương thức `activateDevice()` cho phép người dùng kích hoạt và liên kết thiết bị với tài khoản cá nhân. Trước khi thiết bị được kích hoạt hoặc gửi dữ liệu lên hệ thống, phương thức `validateDevice()` được sử dụng để kiểm tra tính hợp lệ của thông tin thiết bị.

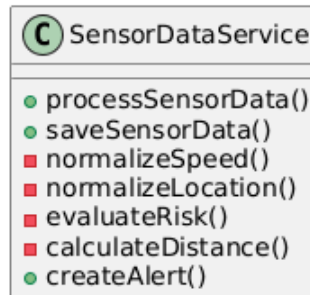
Đối với các chức năng quản lý thiết bị, `DeviceService` cung cấp phương thức `getUserDevices()` để lấy danh sách thiết bị thuộc tài khoản người dùng, `updateDevice()` để cập nhật thông tin thiết bị và `deleteDevice()` để xóa thiết bị khi người dùng không còn nhu cầu quản lý. Phương thức `updateProtectionStatus()` được sử dụng để cập nhật trạng thái bảo vệ của thiết bị, chẳng hạn bật hoặc tắt chế độ chống trộm.

Ngoài các chức năng quản lý, lớp này còn hỗ trợ tương tác với thiết bị IoT. Phương thức `controlBuzzer()` được sử dụng để gửi lệnh điều khiển còi đến thiết bị. Phương thức `getSensorHistory()` cho phép truy xuất lịch sử dữ liệu cảm biến của thiết bị, trong khi `getJourney()` được sử dụng để lấy dữ liệu hành trình hoặc vị trí của phương tiện. Nhờ đó, `DeviceService` đóng vai trò quan trọng trong việc kết nối các chức năng quản lý thiết bị, giám sát trạng thái và điều khiển thiết bị từ xa.

c, Lớp `SensorDataService`

Lớp `SensorDataService` chịu trách nhiệm xử lý dữ liệu cảm biến do thiết bị IoT gửi về hệ thống. Đây là lớp quan trọng trong backend vì dữ liệu cảm biến là cơ sở để cập nhật trạng thái phương tiện, lưu lịch sử hoạt động và phát hiện các dấu hiệu bất thường có thể liên quan đến nguy cơ trộm.

Class SensorDataService



Hình 0.9: Thiết kế lớp `SensorDataService`

Phương thức *processSensorData()* được sử dụng để tiếp nhận và xử lý dữ liệu cảm biến sau khi dữ liệu được gửi từ thiết bị lên server. Trong quá trình xử lý, dữ liệu có thể được chuẩn hóa thông qua các phương thức nội bộ như *normalizeSpeed()* để chuẩn hóa tốc độ và *normalizeLocation()* để chuẩn hóa thông tin vị trí. Sau đó, phương thức *saveSensorData()* thực hiện lưu dữ liệu đã xử lý vào cơ sở dữ liệu để phục vụ việc xem trạng thái và tra cứu lịch sử.

Bên cạnh chức năng lưu trữ, *SensorDataService* còn đảm nhiệm việc đánh giá nguy cơ bất thường. Phương thức *evaluateRisk()* được sử dụng để kiểm tra dữ liệu cảm biến, trạng thái thiết bị và vị trí nhằm xác định có phát sinh nguy cơ trộm hay không. Trong quá trình này, phương thức *calculateDistance()* hỗ trợ tính toán sự thay đổi vị trí của phương tiện, phục vụ việc đánh giá mức độ bất thường khi xe bị di chuyển.

Khi hệ thống xác định dữ liệu có dấu hiệu nguy hiểm, phương thức *createAlert()* được sử dụng để tạo bản ghi cảnh báo trong cơ sở dữ liệu và kích hoạt quá trình gửi thông báo đến người dùng. Nhờ đó, *SensorDataService* đóng vai trò trung tâm trong luồng xử lý dữ liệu cảm biến, từ tiếp nhận, chuẩn hóa, lưu trữ cho đến phát hiện nguy cơ trộm và tạo cảnh báo.

d, Lớp *MqttService*

Lớp *MqttService* chịu trách nhiệm quản lý quá trình giao tiếp giữa backend và thiết bị IoT thông qua giao thức MQTT. Đây là lớp trung gian giúp hệ thống máy chủ nhận dữ liệu từ thiết bị, chuyển dữ liệu cảm biến đến các lớp xử lý nghiệp vụ và gửi lệnh điều khiển ngược lại thiết bị khi cần thiết.

Class MqttService



Hình 0.10: Thiết kế lớp MqttService

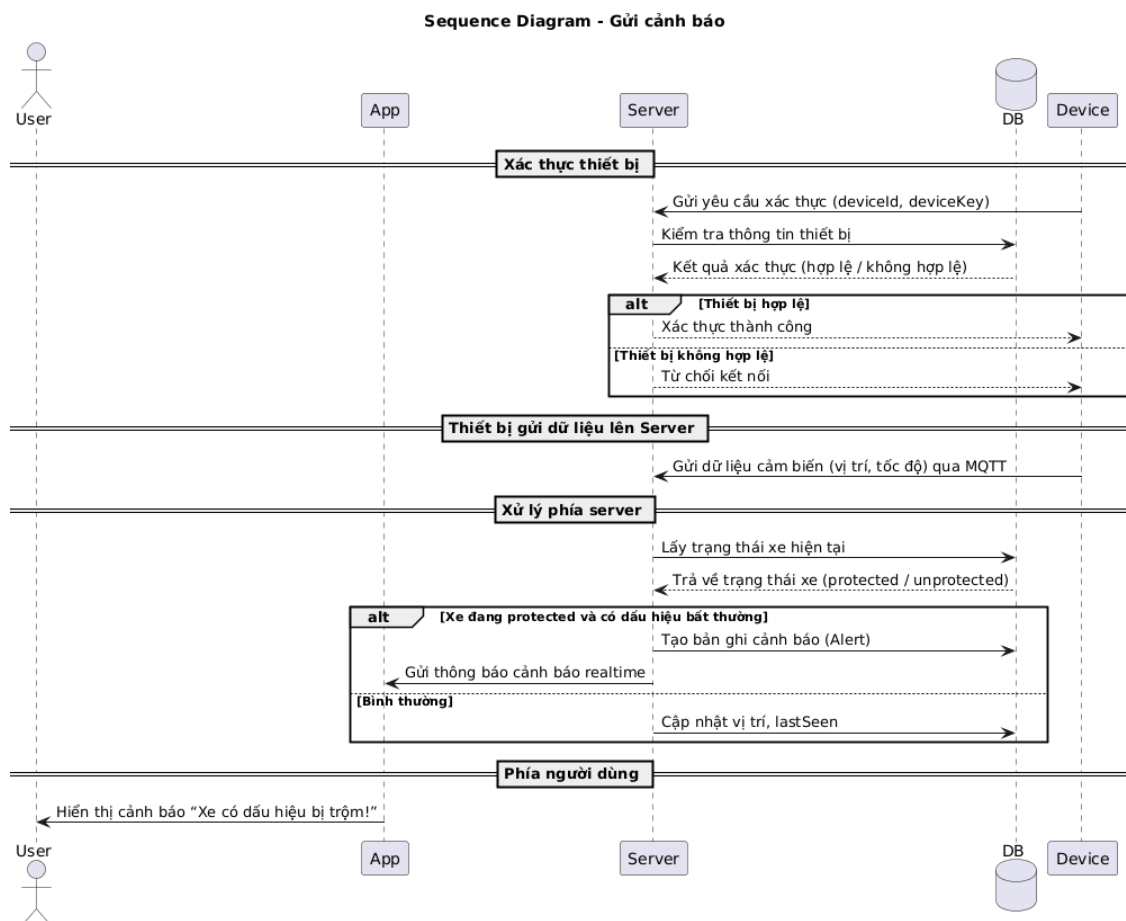
Phương thức *connectToBroker()* được sử dụng để thiết lập kết nối từ backend đến MQTT broker. Sau khi kết nối thành công, phương thức *subscribeToTopics()* thực hiện đăng ký các topic cần lắng nghe để nhận dữ liệu từ thiết bị IoT. Khi thiết bị gửi dữ liệu lên broker, phương thức *receiveMessage()* tiếp nhận bản tin MQTT và phân tích nội dung dữ liệu nhận được.

Trước khi dữ liệu được đưa vào xử lý, phương thức *authenticateDevice()* được sử dụng để kiểm tra tính hợp lệ của thiết bị gửi dữ liệu. Việc xác thực này giúp hạn chế dữ liệu không hợp lệ hoặc dữ liệu từ thiết bị không thuộc hệ thống. Sau khi dữ liệu được xác thực, *MqttService* chuyển dữ liệu đến các lớp xử lý liên quan như *SensorDataService* để lưu trữ, phân tích và phát hiện nguy cơ bất thường.

Ngoài nhiệm vụ nhận dữ liệu, *MqttService* còn hỗ trợ gửi lệnh điều khiển đến thiết bị IoT thông qua phương thức *publishControlCommand()*. Phương thức này được sử dụng trong các chức năng như bật hoặc tắt còi, thay đổi trạng thái bảo vệ hoặc gửi lệnh điều khiển từ ứng dụng di động đến thiết bị. Nhờ đó, *MqttService* đóng vai trò quan trọng trong việc duy trì kênh giao tiếp hai chiều giữa backend service và thiết bị IoT.

e, Biểu đồ tuần tự gửi cảnh báo

Biểu đồ tuần tự gửi cảnh báo mô tả luồng xử lý khi thiết bị IoT gửi dữ liệu cảm biến lên hệ thống và server phát hiện dấu hiệu bất thường cần cảnh báo cho người dùng. Các thành phần tham gia trong biểu đồ gồm người dùng, ứng dụng di động, server, cơ sở dữ liệu và thiết bị.



Hình 0.11: Biểu đồ trình tự Gửi cảnh báo

Trước khi gửi dữ liệu, thiết bị thực hiện xác thực với server bằng cách gửi thông tin *deviceId* và *deviceKey*. Server truy vấn cơ sở dữ liệu để kiểm tra thông tin thiết bị. Nếu thiết bị hợp lệ, server phản hồi xác thực thành công và cho phép thiết bị tiếp tục gửi dữ liệu. Ngược lại, nếu thông tin thiết bị không hợp lệ, server từ chối kết nối hoặc không tiếp nhận dữ liệu từ thiết bị.

Sau khi được xác thực, thiết bị gửi dữ liệu cảm biến lên server thông qua giao thức MQTT. Dữ liệu gửi lên bao gồm các thông tin như vị trí và tốc độ của phương tiện. Khi nhận được dữ liệu, server lấy trạng thái hiện tại của xe từ cơ sở dữ liệu để kiểm tra xem xe đang ở chế độ bảo vệ hay không.

Trong trường hợp xe đang ở trạng thái được bảo vệ và dữ liệu cảm biến có dấu hiệu bất thường, server tạo một bản ghi cảnh báo trong cơ sở dữ liệu. Sau đó, server gửi thông báo cảnh báo theo thời gian thực đến ứng dụng di động. Ứng dụng hiển thị cảnh báo cho người dùng với nội dung xe có dấu hiệu bị trộm, giúp người dùng kịp thời nắm bắt tình huống.

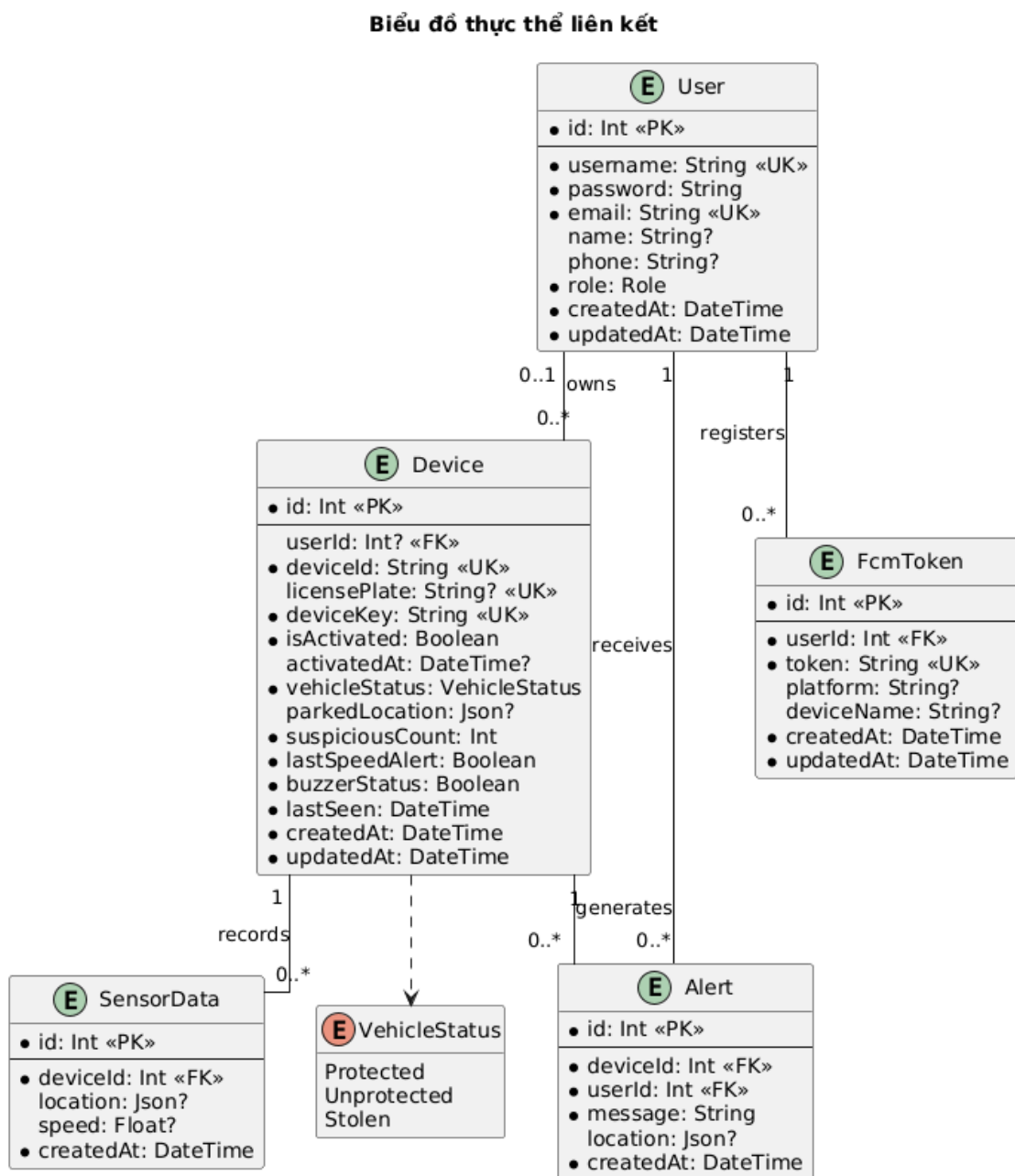
Nếu dữ liệu gửi lên không có dấu hiệu bất thường hoặc xe không ở trạng thái bảo vệ, server chỉ cập nhật thông tin vị trí và thời gian hoạt động gần nhất của thiết

bị. Nhờ luồng xử lý này, hệ thống có thể vừa lưu trữ dữ liệu giám sát, vừa phát hiện và gửi cảnh báo kịp thời khi phương tiện có nguy cơ bị xâm phạm.

0.2.3 Thiết kế cơ sở dữ liệu

a, Biểu đồ thực thể liên kết

Biểu đồ thực thể liên kết mô tả cấu trúc dữ liệu chính của hệ thống chống trộm xe IoT. Các thực thể trong biểu đồ gồm *User*, *Device*, *SensorData*, *Alert*, *FcmToken* và kiểu liệt kê *VehicleStatus*. Các thực thể này được thiết kế nhằm lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến, cảnh báo và token thông báo của ứng dụng di động.



Hình 0.12: Biểu đồ thực thể liên kết

Thực thể *User* lưu trữ thông tin tài khoản người dùng, bao gồm tên đăng nhập, mật khẩu, email, họ tên, số điện thoại, vai trò và thời gian tạo/cập nhật. Trong đó, *username* và *email* là các trường có tính duy nhất để đảm bảo mỗi tài khoản được định danh rõ ràng trong hệ thống.

Thực thể *Device* lưu trữ thông tin thiết bị chống trộm được gắn trên phương tiện. Mỗi thiết bị có mã thiết bị *deviceId*, khóa xác thực *deviceKey*, biển số xe, trạng thái kích hoạt, thời điểm kích hoạt, trạng thái phương tiện, vị trí đỗ xe, trạng thái còi và thời gian hoạt động gần nhất. Trường *userId* cho phép liên kết thiết bị với tài khoản người dùng sau khi thiết bị được kích hoạt.

Thực thể *SensorData* lưu trữ dữ liệu cảm biến do thiết bị gửi lên hệ thống, bao gồm vị trí, tốc độ và thời gian ghi nhận dữ liệu. Mỗi bản ghi dữ liệu cảm biến liên kết với một thiết bị cụ thể thông qua khóa ngoại *deviceId*. Dữ liệu này được sử dụng để hiển thị lịch sử hoạt động, hành trình và hỗ trợ quá trình phát hiện bất thường.

Thực thể *Alert* lưu trữ các cảnh báo được tạo ra khi hệ thống phát hiện dấu hiệu bất thường. Mỗi cảnh báo gắn với một thiết bị và một người dùng, bao gồm nội dung cảnh báo, vị trí liên quan và thời điểm phát sinh. Dữ liệu này phục vụ cho chức năng hiển thị danh sách cảnh báo và lịch sử cảnh báo trên ứng dụng di động.

Thực thể *FcmToken* lưu trữ token thông báo của thiết bị di động. Mỗi token liên kết với một người dùng, có thể kèm theo thông tin nền tảng và tên thiết bị. Thực thể này được sử dụng để hệ thống gửi thông báo cảnh báo đến đúng thiết bị di động của người dùng thông qua Firebase Cloud Messaging.

Các quan hệ trong biểu đồ thể hiện mối liên kết giữa các thực thể. Một người dùng có thể sở hữu nhiều thiết bị, nhận nhiều cảnh báo và đăng ký nhiều FCM token. Một thiết bị có thể sinh ra nhiều bản ghi dữ liệu cảm biến và nhiều cảnh báo. Kiểu liệt kê *VehicleStatus* được sử dụng để biểu diễn trạng thái của phương tiện, gồm các trạng thái *Protected*, *Unprotected* và *Stolen*. Cách thiết kế này giúp dữ liệu trong hệ thống được tổ chức rõ ràng, hỗ trợ tốt cho các chức năng quản lý thiết bị, giám sát dữ liệu và gửi cảnh báo đến người dùng.

b, Thiết kế cơ sở dữ liệu trong PostgreSQL

Từ biểu đồ thực thể liên kết đã trình bày ở phần trên, cơ sở dữ liệu của hệ thống được thiết kế theo mô hình quan hệ và triển khai trên hệ quản trị cơ sở dữ liệu PostgreSQL. Các thực thể trong biểu đồ được ánh xạ thành các bảng dữ liệu, trong đó mỗi bảng có khóa chính để định danh bản ghi và các khóa ngoại để thể hiện quan hệ giữa các bảng.

Trong quá trình triển khai, hệ thống sử dụng Prisma ORM làm lớp trung gian

giữa mã nguồn backend NestJS và PostgreSQL. Các kiểu dữ liệu trong mô hình Prisma được ánh xạ sang các kiểu dữ liệu tương ứng trong PostgreSQL. Ví dụ, kiểu *Int* được ánh xạ sang kiểu số nguyên, *String* được ánh xạ sang kiểu chuỗi, *Boolean* dùng cho dữ liệu đúng/sai, *DateTime* dùng cho dữ liệu thời gian, còn *Json* được sử dụng để lưu các dữ liệu có cấu trúc linh hoạt như vị trí phương tiện.

Các bảng chính trong cơ sở dữ liệu được mô tả trong Bảng 1.

Tên bảng	Mô tả
<i>User</i>	Lưu thông tin tài khoản người dùng, bao gồm tên đăng nhập, email, mật khẩu, thông tin cá nhân và vai trò trong hệ thống.
<i>Device</i>	Lưu thông tin thiết bị chống trộm, mã thiết bị, khóa xác thực, biển số xe, trạng thái kích hoạt, trạng thái phương tiện và thông tin hoạt động gần nhất.
<i>SensorData</i>	Lưu dữ liệu cảm biến do thiết bị IoT gửi lên, bao gồm vị trí, tốc độ và thời gian ghi nhận dữ liệu.
<i>Alert</i>	Lưu thông tin cảnh báo được tạo ra khi hệ thống phát hiện dấu hiệu bất thường liên quan đến phương tiện.
<i>FcmToken</i>	Lưu token thông báo của thiết bị di động, phục vụ chức năng gửi cảnh báo đến người dùng thông qua Firebase Cloud Messaging.

Bảng 1: Danh sách các bảng chính trong cơ sở dữ liệu PostgreSQL

Bảng *User* là bảng trung tâm dùng để lưu trữ thông tin tài khoản người dùng. Mỗi người dùng có một khóa chính *id*. Các trường *username* và *email* được đặt ràng buộc duy nhất để đảm bảo không có hai tài khoản trùng thông tin đăng nhập. Bảng này có quan hệ với bảng *Device*, *Alert* và *FcmToken*.

Bảng *Device* lưu thông tin các thiết bị IoT được sử dụng trong hệ thống. Mỗi thiết bị có một mã định danh *deviceId* và khóa xác thực *deviceKey*. Trường *userId* là khóa ngoại liên kết thiết bị với người dùng sau khi thiết bị được kích hoạt. Ngoài ra, bảng này còn lưu trạng thái phương tiện, vị trí đỗ xe, trạng thái còi, số lần phát hiện bất thường và thời gian thiết bị hoạt động gần nhất.

Bảng *SensorData* lưu các bản ghi dữ liệu được gửi từ thiết bị lên server. Mỗi bản ghi liên kết với một thiết bị thông qua khóa ngoại *deviceId*. Dữ liệu trong bảng này được sử dụng để hiển thị lịch sử cảm biến, hành trình di chuyển và hỗ trợ quá trình phát hiện nguy cơ trộm.

Bảng *Alert* lưu các cảnh báo phát sinh trong hệ thống. Mỗi cảnh báo gắn với

một thiết bị và một người dùng cụ thể thông qua các khóa ngoại *deviceId* và *userId*. Bảng này lưu nội dung cảnh báo, vị trí liên quan và thời điểm phát sinh, phục vụ cho chức năng thông báo và xem lịch sử cảnh báo trên ứng dụng di động.

Bảng *FcmToken* lưu token thông báo của thiết bị di động. Một người dùng có thể có nhiều token khác nhau trong trường hợp đăng nhập trên nhiều thiết bị. Các token này được sử dụng để server gửi cảnh báo đến đúng thiết bị di động của người dùng thông qua Firebase Cloud Messaging.

Các quan hệ chính trong cơ sở dữ liệu gồm: một người dùng có thể sở hữu nhiều thiết bị, một thiết bị có thể sinh ra nhiều bản ghi dữ liệu cảm biến, một thiết bị có thể phát sinh nhiều cảnh báo, và một người dùng có thể đăng ký nhiều FCM token. Việc thiết kế cơ sở dữ liệu theo mô hình quan hệ giúp dữ liệu được tổ chức rõ ràng, đảm bảo tính nhất quán và thuận tiện cho các truy vấn như xem danh sách thiết bị, xem lịch sử dữ liệu, truy xuất cảnh báo và gửi thông báo đến người dùng.

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Trong quá trình xây dựng hệ thống, đề án sử dụng một số công cụ và thư viện phục vụ cho phát triển ứng dụng di động, xây dựng máy chủ, thiết kế cơ sở dữ liệu, lập trình thiết bị IoT và quản lý mã nguồn. Danh sách các công cụ và thư viện chính được trình bày trong Bảng 2.

Mục đích	Công cụ / Thư viện	Địa chỉ URL
Soạn thảo mã nguồn	Visual Studio Code	https://code.visualstudio.com/
Phát triển ứng dụng Android	Android Studio	https://developer.android.com/studio
Phát triển ứng dụng di động	Flutter	https://flutter.dev/
Ngôn ngữ lập trình ứng dụng di động	Dart	https://dart.dev/
Phát triển máy chủ	NestJS	https://nestjs.com/
Môi trường chạy backend	Node.js	https://nodejs.org/
Quản lý package backend	npm	https://www.npmjs.com/
Tương tác cơ sở dữ liệu	Prisma ORM	https://www.prisma.io/
Hệ quản trị cơ sở dữ liệu	PostgreSQL	https://www.postgresql.org/
Phát triển firmware ESP32	Arduino IDE	https://www.arduino.cc/en/software
Nền tảng ESP32	Arduino ESP32 Core	https://github.com/espressif/arduino-esp32
Xử lý dữ liệu JSON trên thiết bị	ArduinoJson	https://arduinojson.org/
Giao tiếp MQTT trên ESP32	PubSubClient	https://pubsubclient.knolleary.net/
Đọc dữ liệu GPS	TinyGPSPlus	https://github.com/mikalhart/TinyGPSPlus
MQTT Broker	HiveMQ Cloud	https://www.hivemq.com/products/mqtt-cloud-broker/
Gửi thông báo đẩy	Firebase Cloud Messaging	https://firebase.google.com/docs/cloud-messaging
Hiển thị bản đồ	OpenStreetMap	https://www.openstreetmap.org/
Quản lý phiên bản mã nguồn	Git	https://git-scm.com/

Bảng 2: Danh sách thư viện và công cụ sử dụng

0.3.2 Kết quả đạt được

Sau quá trình phân tích, thiết kế và triển khai, đồ án đã xây dựng được hệ thống chống trộm xe máy sử dụng thiết bị IoT kết hợp với ứng dụng di động và máy chủ xử lý dữ liệu. Hệ thống bao gồm ba thành phần chính: thiết bị IoT gắn trên xe, ứng dụng di động dành cho người dùng và máy chủ backend. Thiết bị IoT có nhiệm vụ thu thập dữ liệu cảm biến và vị trí của xe, sau đó gửi dữ liệu về hệ thống thông qua

giao thức MQTT. Máy chủ backend tiếp nhận, xử lý, lưu trữ dữ liệu và phát hiện các tình huống bất thường dựa trên tốc độ, khoảng cách di chuyển và trạng thái thiết bị. Ứng dụng di động cho phép người dùng quản lý thiết bị, theo dõi trạng thái xe, xem lịch sử dữ liệu và nhận cảnh báo khi hệ thống phát hiện nguy cơ mất an toàn.

Sản phẩm sau cùng của đề án bao gồm mã nguồn ứng dụng di động, mã nguồn máy chủ backend, cơ sở dữ liệu và chương trình điều khiển thiết bị IoT. Ứng dụng di động được xây dựng nhằm cung cấp giao diện trực quan cho người dùng cuối. Máy chủ backend đóng vai trò trung gian giữa thiết bị IoT, cơ sở dữ liệu và ứng dụng di động. Cơ sở dữ liệu lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến, vị trí và lịch sử cảnh báo. Thiết bị IoT đảm nhận việc thu thập dữ liệu thực tế từ xe và gửi dữ liệu định kỳ về hệ thống.

Thành phần	Kết quả đạt được
Ứng dụng di động	Xây dựng ứng dụng cho phép người dùng đăng nhập, quản lý thiết bị, theo dõi trạng thái xe, xem lịch sử dữ liệu và nhận cảnh báo từ hệ thống.
Máy chủ backend	Xây dựng hệ thống API phục vụ ứng dụng di động, tiếp nhận dữ liệu từ thiết bị IoT, xử lý dữ liệu cảm biến, lưu trữ dữ liệu và gửi cảnh báo đến người dùng.
Thiết bị IoT	Xây dựng chương trình thu thập dữ liệu cảm biến, dữ liệu vị trí và gửi dữ liệu về máy chủ thông qua giao thức MQTT.
Cơ sở dữ liệu	Thiết kế và triển khai cơ sở dữ liệu lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến, vị trí và lịch sử cảnh báo.
Hệ thống cảnh báo	Xây dựng cơ chế phát hiện bất thường dựa trên dữ liệu tốc độ, khoảng cách di chuyển và trạng thái thiết bị, từ đó gửi thông báo cảnh báo đến ứng dụng di động.

Bảng 3: Các thành phần chính đã xây dựng

Ngoài các thành phần chức năng, đề án cũng thống kê một số thông tin kỹ thuật của hệ thống nhằm thể hiện quy mô triển khai. Các thông tin thống kê bao gồm số lượng tệp mã nguồn, số dòng mã, số module chính và dung lượng mã nguồn của từng thành phần. Kết quả thống kê được trình bày trong Bảng 4.

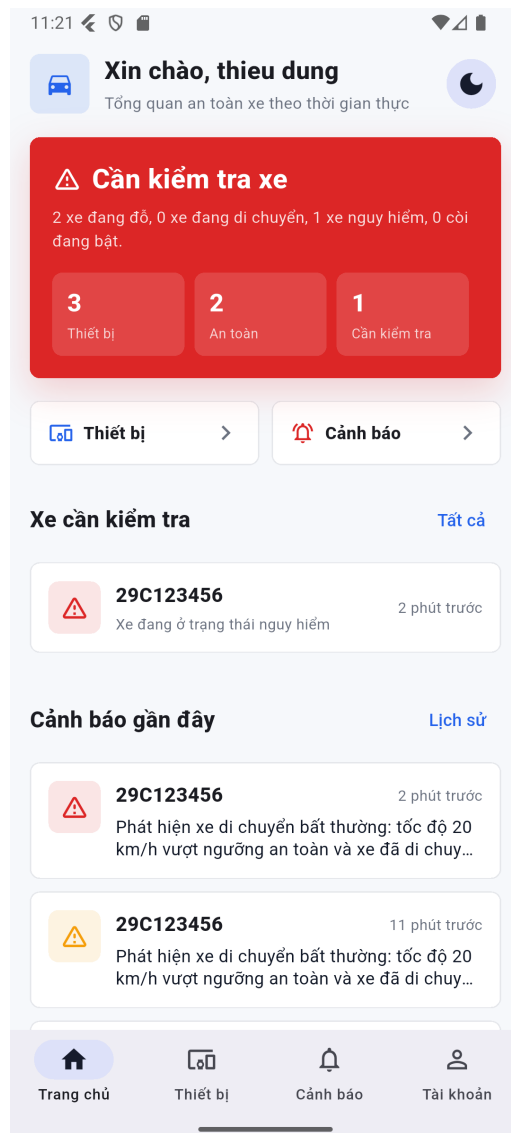
Thành phần	Số tệp mã nguồn	Số dòng mã	Số gói	Dung lượng
Ứng dụng di động	49	≈ 9.000	5	0,27 MB
Máy chủ backend	41	≈ 2.500	9	0,1 MB
Chương trình IoT	1	≈ 1000	1	0,05 MB
Cơ sở dữ liệu	8	≈ 250	1	0,01 MB
Tổng cộng	99	≈ 12.750	16	0,43 MB

Ghi chú: Bảng thống kê chỉ tính các tệp mã nguồn chính do em xây dựng, không bao gồm thư viện phụ thuộc, file build, file cache và các tệp sinh tự động trong quá trình phát triển.

Bảng 4: Thống kê mã nguồn và thành phần triển khai

0.3.3 Minh họa các chức năng chính

a, Màn hình trang chủ

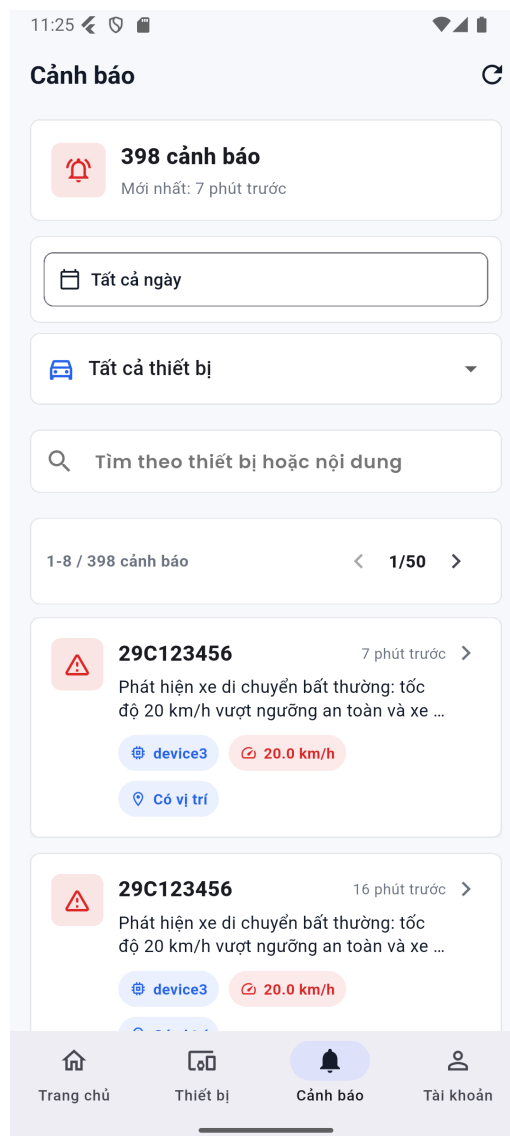


Hình 0.13: Màn hình Trang chủ của ứng dụng

Hình 0.13 trình bày màn hình Trang chủ của ứng dụng. Màn hình này hiển thị tổng quan trạng thái an toàn của các thiết bị, bao gồm số lượng thiết bị, số xe an toàn và số xe cần kiểm tra. Khi có xe ở trạng thái nguy hiểm, hệ thống sử dụng màu đỏ để làm nổi bật cảnh báo, giúp người dùng nhanh chóng nhận biết tình huống bất thường.

Phía dưới màn hình là danh sách xe cần kiểm tra và các cảnh báo gần đây. Mỗi cảnh báo hiển thị biển số xe, thời gian phát sinh và nội dung mô tả ngắn gọn. Màn hình cũng cung cấp các lối tắt đến chức năng quản lý thiết bị và xem cảnh báo, hỗ trợ người dùng thao tác nhanh với các chức năng chính.

b, Màn hình cảnh báo



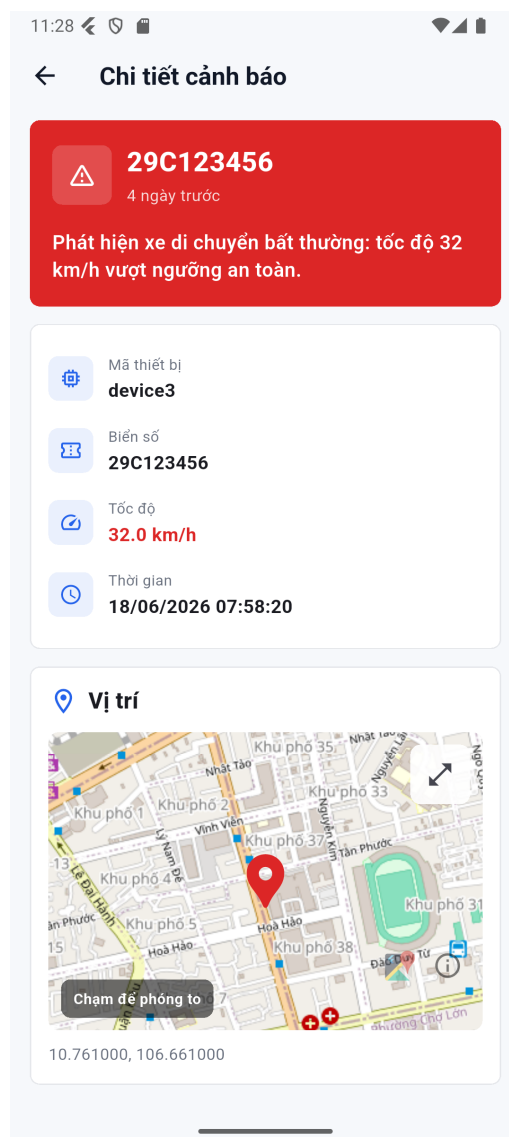
Hình 0.14: Màn hình Cảnh báo của ứng dụng

Hình 0.14 minh họa màn hình Cảnh báo của ứng dụng. Màn hình này hiển thị danh sách các cảnh báo phát sinh từ hệ thống khi phát hiện xe có dấu hiệu di chuyển

bất thường. Phía trên màn hình thể hiện tổng số cảnh báo và thời điểm cảnh báo mới nhất, giúp người dùng nhanh chóng nắm được tình trạng hiện tại.

Ứng dụng cung cấp các bộ lọc theo thời gian, thiết bị và ô tìm kiếm để hỗ trợ người dùng tra cứu cảnh báo thuận tiện hơn. Mỗi mục cảnh báo hiển thị biển số xe, thời gian phát sinh, nội dung cảnh báo, tên thiết bị, tốc độ ghi nhận và trạng thái có vị trí. Nhờ đó, người dùng có thể kiểm tra nhanh thông tin cảnh báo và chuyển sang màn hình chi tiết khi cần xử lý thêm.

c, Màn hình chi tiết cảnh báo



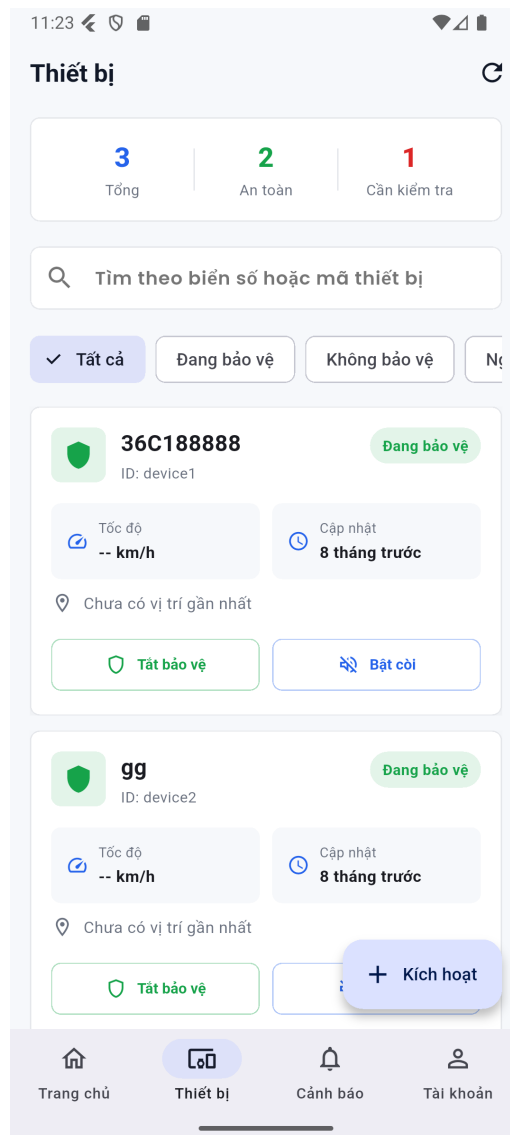
Hình 0.15: Màn hình Chi tiết cảnh báo của ứng dụng

Hình 0.15 minh họa màn hình Chi tiết cảnh báo của ứng dụng. Màn hình này hiển thị đầy đủ thông tin của một cảnh báo cụ thể khi hệ thống phát hiện xe có dấu hiệu di chuyển bất thường. Phần đầu màn hình sử dụng màu đỏ để nhấn mạnh mức độ nghiêm trọng của cảnh báo, đồng thời hiển thị biển số xe, thời điểm phát sinh

và nội dung cảnh báo.

Bên dưới là khu vực thông tin chi tiết, bao gồm mã thiết bị, biển số xe, tốc độ ghi nhận tại thời điểm cảnh báo và thời gian xảy ra sự kiện. Ngoài ra, màn hình còn hiển thị vị trí phát sinh cảnh báo trên bản đồ, kèm theo tọa độ địa lý. Thông tin này giúp người dùng nhanh chóng xác định vị trí của xe và đưa ra phương án xử lý phù hợp.

d, Màn hình thiết bị



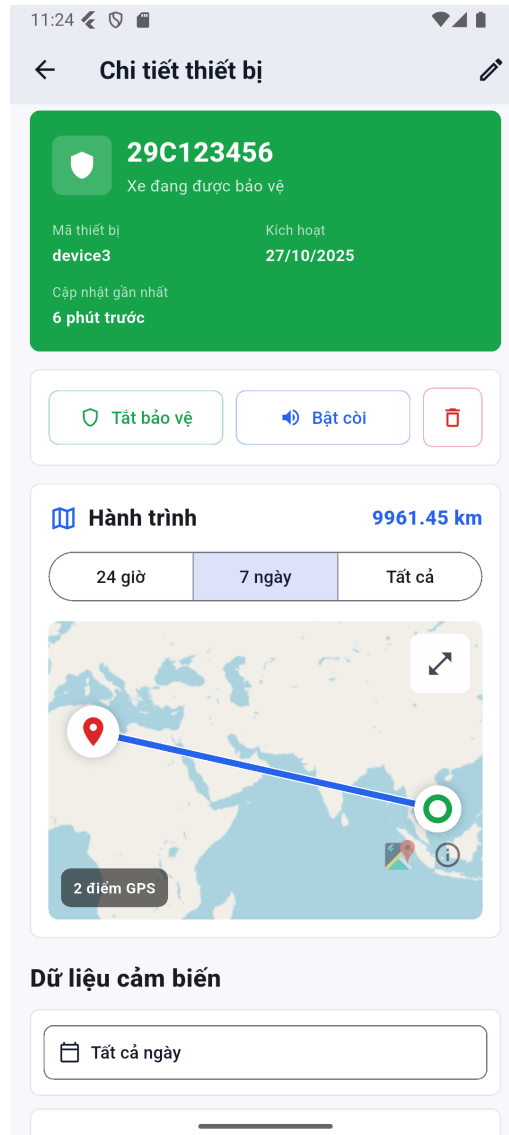
Hình 0.16: Màn hình Thiết bị của ứng dụng

Hình 0.16 minh họa màn hình Thiết bị của ứng dụng. Màn hình này cho phép người dùng theo dõi và quản lý danh sách các thiết bị đã được kích hoạt trong hệ thống. Phần đầu màn hình hiển thị thống kê tổng số thiết bị, số thiết bị đang ở trạng thái an toàn và số thiết bị cần kiểm tra.

Bên dưới là khu vực tìm kiếm và bộ lọc theo trạng thái thiết bị, giúp người dùng

nhANH chóng tra cứu thiết bị theo biển số, mã thiết bị hoặc trạng thái bảo vệ. Mỗi thẻ thiết bị hiển thị các thông tin chính như biển số xe, mã thiết bị, trạng thái bảo vệ, tốc độ gần nhất, thời gian cập nhật và vị trí gần nhất nếu có. Ngoài ra, người dùng có thể thực hiện nhanh các thao tác như bật/tắt chế độ bảo vệ, điều khiển còi hoặc kích hoạt thiết bị mới.

e, Màn hình chi tiết thiết bị



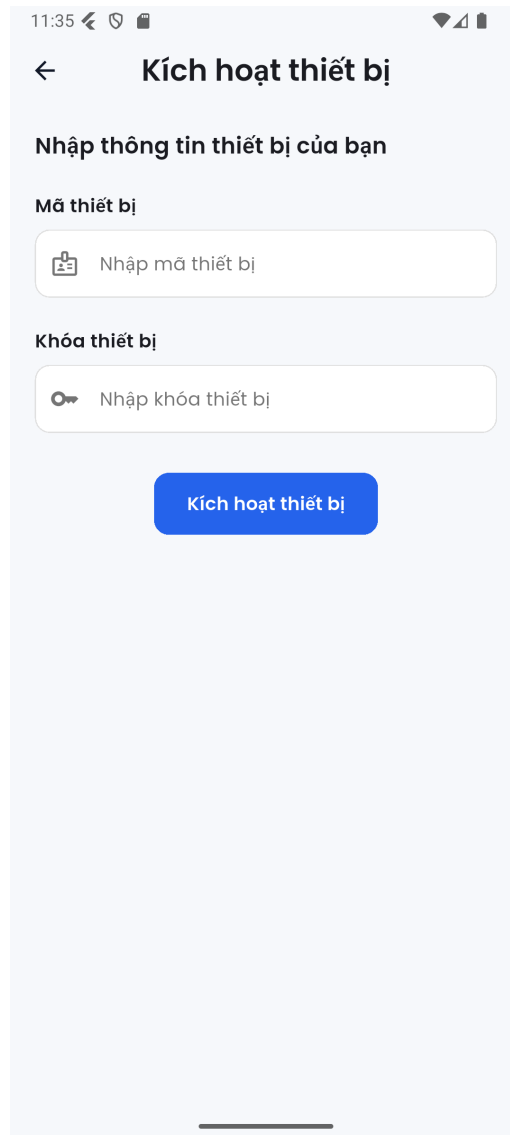
Hình 0.17: Màn hình Chi tiết thiết bị của ứng dụng

Hình 0.17 minh họa màn hình Chi tiết thiết bị của ứng dụng. Màn hình này hiển thị thông tin chi tiết của một thiết bị đang được người dùng quản lý. Phần đầu màn hình thể hiện trạng thái hiện tại của xe, bao gồm biển số, mã thiết bị, ngày kích hoạt và thời gian cập nhật gần nhất. Khi xe đang ở trạng thái an toàn, giao diện sử dụng màu xanh để biểu thị thiết bị đang được bảo vệ.

Bên dưới phần thông tin tổng quan, người dùng có thể thực hiện các thao tác

điều khiển như bật/tắt chế độ bảo vệ, bật còi cảnh báo hoặc xóa thiết bị. Màn hình cũng hiển thị thông tin hành trình của xe trên bản đồ, cho phép người dùng xem quãng đường di chuyển theo các khoảng thời gian như 24 giờ, 7 ngày hoặc toàn bộ dữ liệu. Ngoài ra, phần dữ liệu cảm biến phía dưới hỗ trợ người dùng theo dõi các thông tin thu thập được từ thiết bị IoT.

f, Màn hình kích hoạt thiết bị



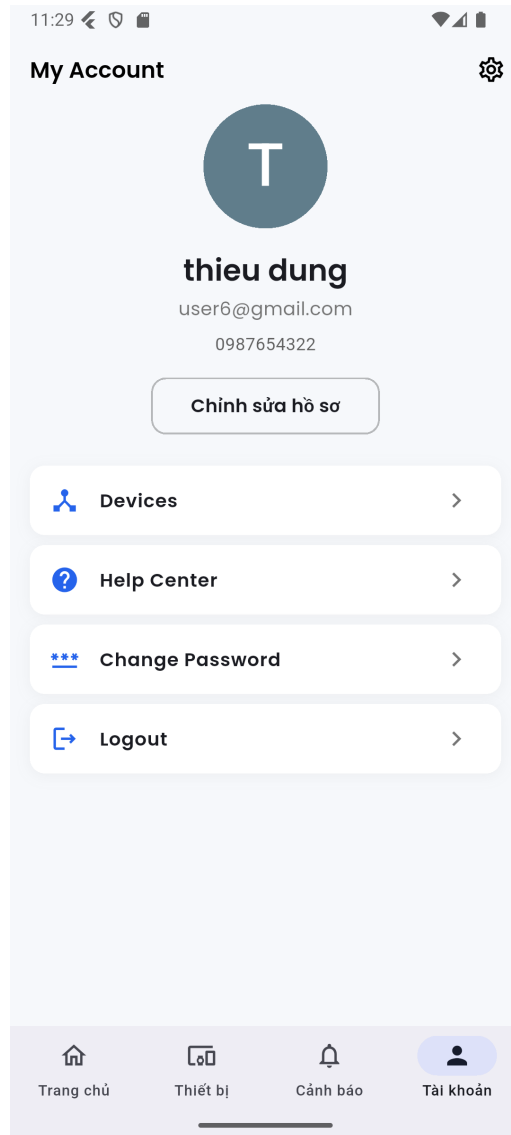
Hình 0.18: Màn hình Kích hoạt thiết bị của ứng dụng

Hình 0.18 minh họa màn hình Kích hoạt thiết bị của ứng dụng. Màn hình này cho phép người dùng liên kết thiết bị IoT với tài khoản cá nhân trước khi sử dụng các chức năng giám sát và cảnh báo. Người dùng cần nhập mã thiết bị và khóa thiết bị được cung cấp tương ứng với thiết bị vật lý.

Sau khi người dùng nhấn nút kích hoạt, ứng dụng gửi thông tin lên máy chủ để kiểm tra tính hợp lệ của thiết bị. Nếu thông tin chính xác và thiết bị chưa được kích

hoạt bởi tài khoản khác, hệ thống sẽ gán thiết bị cho người dùng hiện tại. Chức năng này giúp đảm bảo chỉ người dùng có thông tin xác thực hợp lệ mới có thể quản lý thiết bị trong hệ thống.

g, Màn hình tài khoản cá nhân



Hình 0.19: Màn hình Tài khoản của ứng dụng

Hình 0.19 minh họa màn hình Tài khoản của ứng dụng. Màn hình này hiển thị các thông tin cá nhân cơ bản của người dùng, bao gồm ảnh đại diện, tên, địa chỉ email và số điện thoại. Người dùng có thể truy cập chức năng chỉnh sửa hồ sơ để cập nhật thông tin cá nhân khi cần thiết.

Bên dưới phần thông tin tài khoản là danh sách các chức năng liên quan đến người dùng, bao gồm quản lý thiết bị, trung tâm trợ giúp, đổi mật khẩu và đăng xuất khỏi hệ thống. Cách bố trí này giúp người dùng dễ dàng truy cập các chức năng cá nhân và quản lý tài khoản trong quá trình sử dụng ứng dụng.

0.4 Kiểm thử

Sau khi hoàn thành quá trình xây dựng các thành phần chính của hệ thống, đồ án tiến hành kiểm thử nhằm đánh giá tính đúng đắn của các chức năng quan trọng. Các chức năng được lựa chọn kiểm thử bao gồm kích hoạt thiết bị, phát hiện và gửi cảnh báo, và điều khiển còi cảnh báo. Đây là các chức năng có ảnh hưởng trực tiếp đến quá trình quản lý thiết bị và cảnh báo mất an toàn cho người dùng.

Quá trình kiểm thử chủ yếu sử dụng phương pháp kiểm thử hộp đen, trong đó hệ thống được đánh giá dựa trên dữ liệu đầu vào, điều kiện thực hiện và kết quả đầu ra mong đợi. Ngoài ra, một số nhánh xử lý quan trọng trong hàm đánh giá nguy cơ cũng có thể được kiểm thử theo hướng hộp trắng để đảm bảo bao phủ các trường hợp rẽ nhánh trong quá trình phát hiện bất thường.

0.4.1 Kỹ thuật kiểm thử

Các kỹ thuật kiểm thử được sử dụng trong đồ án được trình bày trong Bảng 5.

Kỹ thuật	Cách áp dụng
Phân vùng tương đương	Chia dữ liệu kiểm thử thành các nhóm hợp lệ, không hợp lệ, không tồn tại và không có quyền truy cập.
Giá trị biên	Kiểm tra các giá trị xung quanh ngưỡng tốc độ 10 km/h, khoảng cách 50 m, khoảng cách nghiêm trọng 100 m và số bản tin liên tiếp.
Bảng quyết định	Kết hợp các điều kiện như trạng thái bảo vệ, tốc độ, khoảng cách di chuyển và quyền sở hữu thiết bị để xác định kết quả xử lý.
Chuyển trạng thái	Kiểm tra sự thay đổi trạng thái của thiết bị, ví dụ từ trạng thái an toàn sang nguy hiểm, hoặc trạng thái còi tắt sang bật và ngược lại.
Đoán lỗi	Kiểm tra các lỗi thường gặp như thiếu JWT, mất kết nối MQTT, action không hợp lệ hoặc token FCM không hợp lệ.

Bảng 5: Các kỹ thuật kiểm thử được sử dụng

0.4.2 Kiểm thử chức năng kích hoạt thiết bị

Chức năng kích hoạt thiết bị cho phép người dùng liên kết thiết bị IoT với tài khoản cá nhân thông qua mã thiết bị và khóa thiết bị. Kỹ thuật kiểm thử chính được sử dụng cho chức năng này là phân vùng tương đương và bảng quyết định. Các ca kiểm thử được trình bày trong Bảng 6.

ID	Dữ liệu/điều kiện	Kết quả mong đợi	Đánh giá
ACT-01	JWT hợp lệ, mã thiết bị và khóa thiết bị đúng, thiết bị chưa được kích hoạt	Kích hoạt thành công; hệ thống lưu userId, cập nhật isActivated=true và activatedAt.	Đạt
ACT-02	Mã thiết bị đúng nhưng khóa thiết bị sai	Trả về lỗi HTTP 400; thông báo mã hoặc khóa thiết bị không chính xác.	Đạt
ACT-03	Mã thiết bị không tồn tại	Trả về lỗi HTTP 400; không thay đổi dữ liệu trong cơ sở dữ liệu.	Đạt
ACT-04	Thiết bị đã thuộc về một người dùng khác	Trả về lỗi HTTP 409; thông báo thiết bị đã được kích hoạt.	Đạt
ACT-05	Không gửi JWT	Trả về lỗi HTTP 401; không kích hoạt thiết bị.	Đạt
ACT-06	Thiếu mã thiết bị hoặc khóa thiết bị	Yêu cầu bị từ chối; không thay đổi dữ liệu trong cơ sở dữ liệu.	Đạt

Bảng 6: Các ca kiểm thử chức năng kích hoạt thiết bị

Kết quả kiểm thử cho thấy chức năng kích hoạt thiết bị hoạt động đúng với các trường hợp đã thiết kế. Hệ thống cho phép kích hoạt khi thông tin hợp lệ và từ chối các yêu cầu sai khóa, thiếu thông tin, thiếu JWT hoặc thiết bị đã được kích hoạt. Các trường hợp lỗi đều không làm thay đổi dữ liệu trong cơ sở dữ liệu, qua đó đảm bảo tính đúng đắn và an toàn của chức năng.

0.4.3 Kiểm thử chức năng phát hiện và gửi cảnh báo

Chức năng phát hiện và gửi cảnh báo là chức năng quan trọng của hệ thống. Chức năng này xử lý dữ liệu cảm biến gửi từ thiết bị IoT, đánh giá nguy cơ bất thường dựa trên tốc độ, khoảng cách di chuyển và trạng thái bảo vệ của xe, sau đó tạo cảnh báo nếu thỏa mãn điều kiện. Các kỹ thuật kiểm thử được sử dụng bao gồm giá trị biên, bảng quyết định và chuyển trạng thái.

ID	Bảo vệ	Dữ liệu gửi liên tiếp	Kết quả mong đợi	Đánh giá
ALT-01	Tắt	Tốc độ 30 km/h, gửi 2 lần	Không tạo cảnh báo.	Đạt
ALT-02	Bật	Tốc độ 10 km/h, khoảng cách 50 m	Không tạo cảnh báo.	Đạt
ALT-03	Bật	Tốc độ 10,01 km/h, gửi 1 lần	Tăng bộ đếm bất thường lên 1, chưa tạo cảnh báo.	Đạt
ALT-04	Bật	Tốc độ 10,01 km/h, gửi 2 lần	Tạo cảnh báo mức trung bình.	Đạt
ALT-05	Bật	Khoảng cách 50,01 m, gửi 2 lần	Tạo cảnh báo mức trung bình.	Đạt
ALT-06	Bật	Khoảng cách 100,01 m, gửi 2 lần	Đánh dấu xe ở trạng thái nguy hiểm và gửi cảnh báo khẩn cấp.	Đạt
ALT-07	Bật	Tốc độ lớn hơn 10 km/h và khoảng cách lớn hơn 50 m, gửi 2 lần	Đánh dấu xe ở trạng thái nguy hiểm.	Đạt
ALT-08	Bật	Bất thường, bình thường, bất thường	Bộ đếm được đặt lại, chưa tạo cảnh báo.	Đạt
ALT-09	Bật	Gửi thêm dữ liệu sau khi đã tạo cảnh báo	Không tạo cảnh báo trùng lặp.	Đạt
ALT-10	Bật	3 lần liên tiếp vượt ngưỡng tốc độ, khoảng cách bình thường	Kiểm tra khả năng chuyển sang trạng thái nguy hiểm theo ngưỡng 3 lần liên tiếp.	Đạt

Bảng 7: Các ca kiểm thử chức năng phát hiện và gửi cảnh báo

Kết quả kiểm thử cho thấy chức năng phát hiện và gửi cảnh báo hoạt động đúng với các trường hợp đã thiết kế. Hệ thống không tạo cảnh báo khi xe không ở trạng thái bảo vệ hoặc dữ liệu chưa vượt ngưỡng. Khi dữ liệu bất thường vượt ngưỡng và xuất hiện đủ số lần liên tiếp, hệ thống tạo cảnh báo tương ứng và cập nhật trạng thái nguy hiểm khi cần thiết. Các trường hợp đặt lại bộ đếm và tránh cảnh báo trùng lặp cũng được xử lý đúng, góp phần đảm bảo tính chính xác của chức năng cảnh báo.

0.4.4 Kiểm thử chức năng điều khiển còi

Chức năng điều khiển còi cho phép người dùng bật hoặc tắt còi cảnh báo trên thiết bị IoT. Chức năng này cần kiểm tra quyền sở hữu thiết bị, tính hợp lệ của hành động điều khiển và khả năng gửi lệnh qua MQTT. Các kỹ thuật kiểm thử được sử dụng bao gồm phân vùng tương đương, chuyển trạng thái và đoán lỗi.

ID	Điều kiện	Kết quả mong đợi	Đánh giá
BUZ-01	Chủ xe gửi action=on	Publish lệnh MQTT tới topic điều khiển còi; cơ sở dữ liệu lưu trạng thái true.	Đạt
BUZ-02	Chủ xe gửi action=off	Publish lệnh MQTT tới topic điều khiển còi; cơ sở dữ liệu lưu trạng thái false.	Đạt
BUZ-03	Người dùng không sở hữu thiết bị	Trả về lỗi HTTP 403; không publish lệnh MQTT.	Đạt
BUZ-04	Thiết bị không tồn tại	Yêu cầu thất bại; không publish lệnh MQTT.	Đạt
BUZ-05	Không gửi JWT	Trả về lỗi HTTP 401.	Đạt
BUZ-06	action khác on hoặc off	Trả về lỗi HTTP 400; không thay đổi dữ liệu trong cơ sở dữ liệu.	Đạt
BUZ-07	MQTT broker mất kết nối	Trả về lỗi; trạng thái trong cơ sở dữ liệu không được cập nhật sai.	Đạt

Bảng 8: Các ca kiểm thử chức năng điều khiển còi

Kết quả kiểm thử cho thấy chức năng điều khiển còi xử lý đúng các trường hợp bật, tắt còi và các tình huống lỗi. Hệ thống chỉ gửi lệnh MQTT khi người dùng có quyền sở hữu thiết bị và action hợp lệ. Với các yêu cầu thiếu xác thực, sai quyền, thiết bị không tồn tại hoặc action không hợp lệ, hệ thống đều từ chối xử lý và không làm thay đổi dữ liệu. Điều này đảm bảo chức năng điều khiển còi hoạt động đúng và an toàn.

0.4.5 Công cụ kiểm thử

Các công cụ được sử dụng trong quá trình kiểm thử được trình bày trong Bảng 9.

Công cụ	Mục đích sử dụng
Jest 29.7.0	Thực hiện unit test cho các service xử lý nghiệp vụ.
NestJS Testing 11.1.7	Tạo testing module để kiểm thử các thành phần trong backend NestJS.
Supertest 7.1.4	Kiểm thử các API HTTP của hệ thống.
Mock Prisma, MQTT, Socket.IO và Firebase	Giả lập các thành phần bên ngoài để kiểm tra lời gọi và kết quả xử lý mà không phụ thuộc vào hệ thống thật.
Android Emulator	Kiểm thử giao diện và thao tác người dùng trên ứng dụng Flutter.

Bảng 9: Các công cụ kiểm thử sử dụng trong đồ án

Các file kiểm thử chính dự kiến được xây dựng gồm `device.service.spec.ts`, `sensor-data.service.spec.ts` và `mqtt.controller.spec.ts`. Trong đó, `device.service.spec.ts` tập trung kiểm thử các chức năng kích hoạt thiết bị và điều khiển thiết bị; `sensor-data.service.spec.ts` kiểm thử logic xử lý dữ liệu cảm biến và phát hiện cảnh báo; còn `mqtt.controller.spec.ts` kiểm thử các luồng nhận dữ liệu từ thiết bị IoT.

0.4.6 Tổng kết kiểm thử

Nhìn chung, các ca kiểm thử được thiết kế nhằm bao phủ các chức năng quan trọng nhất của hệ thống, bao gồm liên kết thiết bị với tài khoản người dùng, phát hiện bất thường từ dữ liệu cảm biến và điều khiển còi cảnh báo. Việc kết hợp nhiều kỹ thuật kiểm thử giúp hệ thống được đánh giá trong cả trường hợp dữ liệu hợp lệ, dữ liệu không hợp lệ, giá trị biên và các tình huống lỗi thường gặp. “

0.5 Triển khai

Sau khi hoàn thành quá trình xây dựng và kiểm thử, hệ thống được triển khai thử nghiệm nhằm đánh giá khả năng phối hợp giữa các thành phần gồm thiết bị IoT, máy chủ backend, cơ sở dữ liệu, dịch vụ thông báo và ứng dụng di động. Việc triển khai tập trung vào kiểm tra luồng truyền dữ liệu từ thiết bị gắn trên xe về máy chủ, khả năng xử lý dữ liệu cảm biến, phát hiện cảnh báo và hiển thị thông tin cho người dùng trên ứng dụng di động.

0.5.1 Mô hình triển khai

Hệ thống được triển khai theo mô hình gồm thiết bị IoT, máy chủ backend, cơ sở dữ liệu PostgreSQL, MQTT broker, Firebase Cloud Messaging và ứng dụng di động. Thiết bị IoT được gắn trên xe, sử dụng ESP32 kết hợp với module GPS và module SIM để thu thập dữ liệu vị trí, tốc độ và gửi dữ liệu về hệ thống thông qua

giao thức MQTT. MQTT broker đóng vai trò trung gian truyền nhận dữ liệu giữa thiết bị IoT và máy chủ backend.

Máy chủ backend được xây dựng bằng NestJS, đảm nhiệm vai trò tiếp nhận dữ liệu từ thiết bị, xử lý logic nghiệp vụ, lưu trữ dữ liệu vào PostgreSQL và gửi cảnh báo đến người dùng khi phát hiện tình huống bất thường. Bên cạnh đó, backend cũng cung cấp các API phục vụ ứng dụng di động, hỗ trợ các chức năng như đăng nhập, kích hoạt thiết bị, quản lý thiết bị, xem lịch sử dữ liệu và điều khiển còi cảnh báo.

Ứng dụng di động được xây dựng bằng Flutter và cài đặt thử nghiệm trên thiết bị Android. Ứng dụng cho phép người dùng theo dõi trạng thái xe, xem danh sách cảnh báo, xem vị trí xe trên bản đồ và thực hiện các thao tác điều khiển thiết bị. Dịch vụ Firebase Cloud Messaging được sử dụng để gửi thông báo cảnh báo đến điện thoại của người dùng khi hệ thống phát hiện xe có dấu hiệu di chuyển bất thường.

0.5.2 Quy trình triển khai

Quá trình triển khai được thực hiện theo từng bước. Trước hết, cơ sở dữ liệu PostgreSQL được cấu hình và khởi tạo lược đồ dữ liệu thông qua Prisma. Các bảng dữ liệu chính bao gồm thông tin người dùng, thiết bị, dữ liệu cảm biến, cảnh báo và token nhận thông báo. Sau đó, máy chủ backend được cấu hình các thông tin cần thiết như chuỗi kết nối cơ sở dữ liệu, địa chỉ MQTT broker và thông tin xác thực Firebase Cloud Messaging.

Tiếp theo, chương trình điều khiển được nạp vào thiết bị ESP32. Trong chương trình này, thiết bị được cấu hình mã thiết bị, khóa thiết bị và thông tin kết nối tới MQTT broker. Sau khi khởi động, thiết bị thực hiện kết nối mạng, lấy dữ liệu vị trí từ module GPS và gửi dữ liệu cảm biến về máy chủ theo định dạng đã thiết kế.

Ở phía ứng dụng di động, người dùng cài đặt ứng dụng trên thiết bị Android, đăng ký hoặc đăng nhập tài khoản, sau đó kích hoạt thiết bị bằng mã thiết bị và khóa thiết bị. Khi thiết bị đã được liên kết với tài khoản, người dùng có thể bật chế độ bảo vệ, theo dõi trạng thái xe và nhận cảnh báo nếu hệ thống phát hiện dữ liệu bất thường.

0.5.3 Kết quả triển khai thử nghiệm

Trong phạm vi đề án, hệ thống được triển khai thử nghiệm với quy mô nhỏ nhằm đánh giá khả năng phối hợp giữa thiết bị IoT, máy chủ backend, cơ sở dữ liệu, MQTT broker, Firebase Cloud Messaging và ứng dụng di động. Quy mô thử nghiệm dự kiến gồm 5 người dùng, 3 thiết bị trên hệ thống. Mỗi người dùng tương

ứng với một FCM token để kiểm tra khả năng gửi thông báo cảnh báo đến ứng dụng di động. Thời gian thử nghiệm được ước tính trong 7 ngày.

Trong quá trình thử nghiệm, hệ thống phát sinh khoảng 1.000 lượt truy cập API, bao gồm các thao tác đăng nhập, lấy danh sách thiết bị, xem lịch sử cảnh báo, lấy dữ liệu cảm biến, xem hành trình, bật/tắt chế độ bảo vệ và gửi lệnh điều khiển còi. Với chu kỳ gửi dữ liệu cảm biến là 15 giây, một thiết bị nếu chạy liên tục có thể tạo ra khoảng 6.000 bản ghi mỗi ngày. Trong kịch bản thử nghiệm thực tế, thiết bị hoạt động trung bình khoảng 4 giờ mỗi ngày, tương ứng khoảng 1000 bản ghi cảm biến mỗi ngày. Sau 7 ngày thử nghiệm, số lượng bản ghi cảm biến ước tính đạt khoảng 6.500 bản ghi. Tổng dung lượng cơ sở dữ liệu trong phạm vi thử nghiệm dao động khoảng 10–15 MB, bao gồm dữ liệu người dùng, thiết bị, dữ liệu cảm biến, cảnh báo và token thông báo.

Về chức năng cảnh báo, hệ thống phát sinh khoảng 30–50 cảnh báo trong quá trình thử nghiệm. Các cảnh báo được tạo ra khi thiết bị đang ở trạng thái bảo vệ và dữ liệu gửi về vượt các ngưỡng bất thường đã thiết kế. Sau khi cảnh báo được tạo, backend lưu thông tin cảnh báo vào cơ sở dữ liệu, cập nhật trạng thái thiết bị và gửi thông báo đến ứng dụng di động thông qua Firebase Cloud Messaging. Toàn bộ luồng cảnh báo, tính từ thời điểm thiết bị gửi dữ liệu bất thường đến khi người dùng nhận được thông báo, có thời gian phản hồi ước tính khoảng 1–2,5 giây trong điều kiện thông thường và khoảng 4 giây ở mức P95.

Thời gian phản hồi của các chức năng chính ở mức phù hợp với phạm vi thử nghiệm. Các API đơn giản như lấy danh sách thiết bị, bật/tắt chế độ bảo vệ hoặc gửi lệnh còi có thời gian phản hồi trung bình trong khoảng 50–80 ms. Các chức năng cần truy vấn nhiều dữ liệu hơn như lấy dữ liệu cảm biến, lấy lịch sử cảnh báo hoặc lấy hành trình có thời gian phản hồi trung bình khoảng 70–120 ms. Đối với giao tiếp thời gian thực, Socket.IO có độ trễ ước tính khoảng 50–150 ms trong điều kiện thông thường. Dữ liệu MQTT từ ESP32 đến backend có độ trễ khoảng 100–300 ms, trong khi thông báo đẩy qua Firebase Cloud Messaging thường mất khoảng 0,5–2 giây để đến thiết bị người dùng.

Hệ thống cũng được ước tính khả năng chịu tải ở các mức người dùng đồng thời khác nhau. Với 1 đến 10 người dùng đồng thời, thời gian phản hồi trung bình dao động từ khoảng 80 ms đến 120 ms và chưa ghi nhận lỗi đáng kể. Khi tăng lên 25 người dùng đồng thời, thời gian phản hồi trung bình khoảng 220 ms, P95 khoảng 450 ms và tỷ lệ lỗi ước tính khoảng 0,5%. Ở mức 50 người dùng đồng thời, thời gian phản hồi trung bình có thể tăng lên khoảng 450 ms, P95 khoảng 900 ms và tỷ lệ lỗi ước tính khoảng 2–3%. Kết quả này cho thấy hệ thống đáp ứng được quy

mô thử nghiệm nhỏ, nhưng cần được tối ưu thêm nếu triển khai với số lượng người dùng và thiết bị lớn hơn.

Đối với luồng MQTT, với chu kỳ gửi dữ liệu 15 giây, 1 thiết bị tạo ra khoảng 0,07 bản tin mỗi giây. Khi mở rộng lên 10 thiết bị, lưu lượng ước tính đạt khoảng 0,67 bản tin mỗi giây; với 50 thiết bị là khoảng 3,33 bản tin mỗi giây; và với 100 thiết bị là khoảng 6,67 bản tin mỗi giây. Mức lưu lượng này chưa quá lớn đối với hệ thống backend thử nghiệm, tuy nhiên khi triển khai thực tế cần bổ sung cơ chế giám sát MQTT broker, hàng đợi xử lý dữ liệu và tối ưu ghi dữ liệu vào cơ sở dữ liệu để tránh nghẽn khi số lượng thiết bị tăng cao.